

PROJECT REPORT: HELPMATE AI PROJECT

Retrieval Augmented Generation with LlamaIndex

1. Problem Statement:

We are building a project in the insurance domain. The goal of the project will be to build a robust generative search system capable of effectively and accurately answering questions from various insurance policy documents.

LlamaIndex, formerly known as GPT Index, is a data framework designed to facilitate the integration, structuring and access to private or domain-specific data for applications that utilise large language models (LLMs), such as GPT-3 or similar systems. It is a Python library for building data-aware applications and question-answering systems.

2. Why Llama Index:

LlamaIndex supports ingesting data from various sources like text files, web pages, PDFs, and even structured data like CSV files. Once data is ingested, it builds an index that allows for efficient retrieval and querying of the data. It provides a query engine that enables users to ask natural language questions and receive relevant answers from the indexed data. It is an ideal framework as it offers customizable retrieval strategies, such as vector similarity search, keyword search, and more, to find the most relevant data for a given query.

3. Objectives:

The primary objectives of the project are as follows:

- Build a robust generative search system capable of effectively and accurately answering questions from various insurance policy documents.
- Use LangChain or LlamaIndex to build the generative search application.

4. Design

Augmented Generation (RAG):

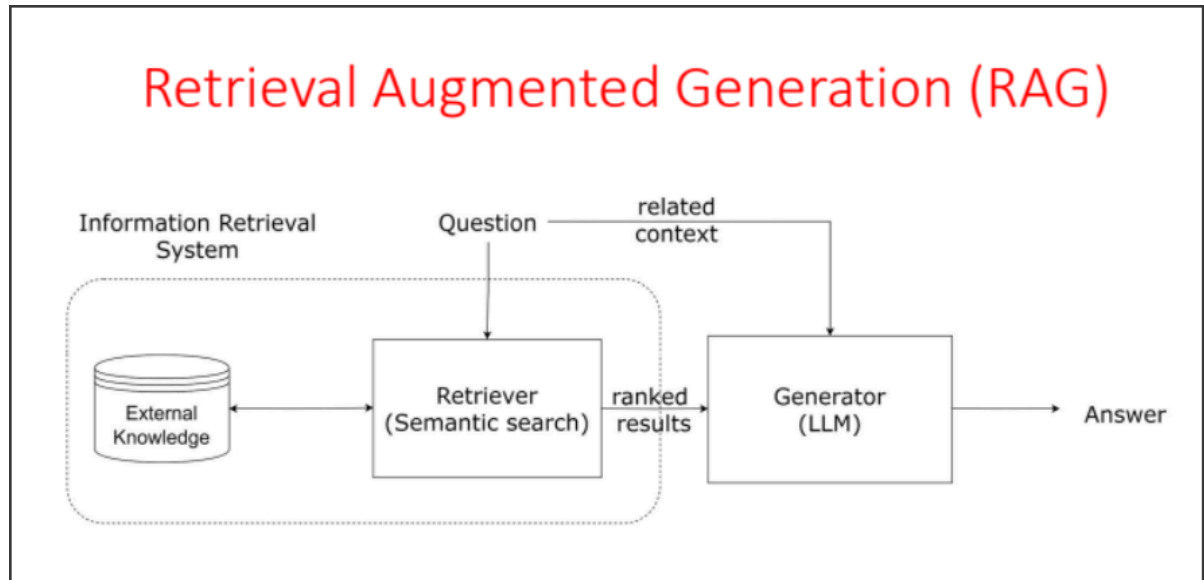
This architecture combines a retriever module (for fetching relevant information) with a generator module (to generate final answers using large language models or LLMs). Below is a detailed step-by-step explanation of the process:

Steps in RAG 1. Question Input a question or query is provided by the user.

This question acts as the input to the system.

The question Summary of Key Components in RAG Question: Input query from the user.

RAG System Architecture



Retriever:

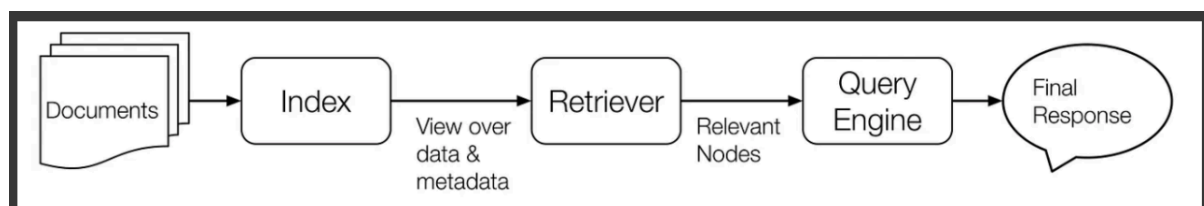
Retrieves and ranks relevant knowledge from external sources using semantic search.

Generator:

Combines the input query and retrieved context to generate a coherent and factually accurate response.

Answer:

Final natural language output provided to the user.



Components:

- Data Loading
- Building query engine
- Creating a Response Pipeline
- Build a Testing Pipeline
- Optimization and fine tuning

5. Implementation

Data Loading:

Used the Simple Directory Reader and read the data from the PDFs given as source data and load the data into a variable.

```
[ ] # Import the necessary Reader
    from llama_index.core import SimpleDirectoryReader

    # Let us take input from a directory
    reader = SimpleDirectoryReader(input_dir="/content/drive/MyDrive/Colab Notebooks/Policy+Documents/")

    # Use the load_data() method to read the files from the directory
    documents2 = reader.load_data()
    # number of files
    print(f"Loaded {len(documents2)} docs")
```

Loaded 217 docs

Building query engine:

A Query Engine wraps a Retriever and a ResponseSynthesizer into a pipeline. It uses the query string to fetch nodes and then sends them to the Large Language Model (LLM) to generate a response.

```
[ ] # Import the necessary libraries
    from llama_index.core.node_parser import SimpleNodeParser
    from llama_index.core import VectorStoreIndex
    from IPython.display import display, HTML

    # create parser and parse document into nodes
    parser = SimpleNodeParser.from_defaults()
    nodes = parser.get_nodes_from_documents(documents2)

    # # build index
    index = VectorStoreIndex(nodes)

    # Construct Query Engine
    query_engine = index.as_query_engine()
    # Perform query operation and generate the response
    query_response = query_engine.query("What are all the conditions for termination of insurance in the poorna suraksha scheme?")
```

Creating a response pipeline:

This involves a query response function and initialize_conv function. A Query Response pipeline encapsulates all the necessary steps to build a RAG pipeline and the initialize_conv() function creates an interactive chatbot.

```
response = query_engine.query(user_input)
file_name = response.source_nodes[0].node.metadata['file_name'] + "page nos: " + response.source_nodes[0].node.metadata['page_label'] + ", " + response.source_nodes[1].node.metadata['page_l
final_response = response.response + '\n Check further at ' + file_name
return final_response
```

```
def initialize_conv():
    """
    Initialize a conversation with the user, allowing them to ask questions
    about the policy documents. The user can type 'exit' to end the
    conversation.

    The function continuously prompts the user for input, processes the input
    using the query_response function, and displays the response. The loop
    terminates when the user types 'exit'.
    """
    print('Feel free to ask Questions regarding HDFC insurance plans. Press exit once you are done')
    while True:
        user_input = input()
        # Type 'exit' to exit conversation
        if user_input.lower() == 'exit':
            print('Exiting the program... bye')
            break
        else:
            response = query_response(user_input)
            display(HTML(f'<p style="font-size:20px">{response}</p>'))
```

Build a Testing Pipeline:

Created 3 questions and store them in the questions list to be queried by the RAG system using the testing_pipeline function.

Optimization and fine tuning:

Based on the testing pipeline's feedback, we can improve the performance by building a custom prompt template, Using customised notes and templates, Sub question query engine, Build RAG using Agentic Systems

6. Challenges:

1. Data Extraction from PDFs - Used Simple DirectoryReader to extract all the PDFs from the folder directly
2. Indexing Large Documents - Use efficient indexing strategies and consider distributed or incremental indexing if the dataset is extremely large.
3. Handling Domain-Specific Language - Fine-tune the language model on a corpus of insurance-related texts to improve its understanding and generation capabilities¹
4. Ensuring Relevant Document Retrieval - Optimize the retrieval mechanism using advanced techniques like SentenceSplitter from llama.index.core.node_parser
5. Combining Retrieved Documents and Query - Develop a structured approach to concatenate the query and retrieved documents, ensuring the input remains within the model's maximum token limit.

7. Lessons Learnt:

1. Importance of Robust Data Extraction
2. Effective Data Preprocessing is Crucial

3. Retrieval Mechanism Optimization
4. Managing Input Length Constraints
5. Evaluation is Multifaceted
6. Handling Sensitive Information